Sprint 3 — Relatório de análise crítica da suíte gerada por LLM

Projeto P07 — Controle de Frequência · Branch `sprint3/testes` · Medições de 09/11/2026

1. Contexto

- **Stack:** NestJS, TypeScript, Prisma, Vitest 4.1.11 e cobertura V8. Testes com mocks do Prisma.
- **User stories (4):** US01 abrir e administrar a chamada temporária; US02 registrar presença com o código; US03 consultar chamadas disponíveis; US04 classificar o risco de reprovação por faltas. Somam **21 critérios de aceitação** (CA01.1 a CA04.4), reconstruídos a partir do enunciado e do código com apoio do ChatGPT, pois não havia documento prévio de user stories.
- **LLMs:** a **v1** foi gerada pelo Claude Opus 5.5 em 09/11/2026, numa conversa nova e sem a matriz. A **v2** vem do ChatGPT (GPT-6), de 08/11/2026.
- **Escopo medido:** `chamada-codigo.service.ts`, os dois DTOs da chamada e `frequencia.util.ts`. Os testes antigos de `src/**/*.spec.ts` ficam fora da medição.

2. Projeto manual

A matriz de rastreabilidade (`docs/testes/matriz-rastreabilidade.md`) lista **21 critérios, 25 partições de equivalência, 16 valores-limite e 7 fluxos**, que geram **48 casos de teste (T01–T48)**. Ela foi commitada em `e2866be7`, antes de qualquer suíte.

3. Geração

- **v1:** um único prompt (`docs/testes/prompt-v1.md`) com as técnicas exigidas, o texto dos 21 critérios, o código dos quatro arquivos e o `schema.prisma`. O prompt não contém a matriz (0 ocorrências). Foi **uma rodada, sem nenhuma correção**: 117 testes, 117 passando, nenhum nome ou caminho inventado.
- **v2:** construída em 08/11 a partir da matriz de 48 casos, antes de existir a v1 (`docs/testes/llm-log.md`, Parte 2).
- **Ressalva:** o prompt pediu explicitamente as técnicas e trouxe os critérios completos, então a amplitude da v1 não deve ser atribuída só ao modelo.
- **Limite do protocolo:** não foi verificado se a memória do aplicativo estava ativa na conversa da v1.

4. Indicadores

Indicador	v1	v2
Testes (passam / falham / não compilam)	**117 (117/0/0)**	48 (48/0/0)
Cobertura de linhas	**100%**	**100%**
Cobertura de branches	**97,91%**	**97,91%**
Cobertura de requisitos	21/21	21/21
Partições / limites / fluxos	25/25 · 16/16 · 7/7	25/25 · 16/16 · 7/7
Testes sem asserção	0	0
Oráculo genérico (`rejects.toThrow()` sem o erro)	**15**	**0**
Defeitos reais encontrados	0	0

As duas suítes foram medidas no mesmo código e no mesmo escopo (`indicadores-v1-v2.md`; relatórios HTML em `docs/testes/relatorios/`).

5. Análise crítica

5.1 Técnicas funcionais: a LLM achou as mesmas partições e limites?

A v1 exercitou **todos** os 48 casos da matriz (25/25 partições, 16/16 limites, 7/7 fluxos); não perdeu nenhum.

Além dos 48 casos, a v1 trouxe **69 testes que a matriz não previa**, e nenhum contradiz o requisito (todos passam). Exemplos que a equipe não tinha listado: e-mail vazio (L463), duração negativa (L170) e do tipo texto (L190), código com espaço interno (L583), com dígito (L578) e com letra acentuada (L589), 1 ms depois da expiração (L754), 15,1%, 24,9% e 100% de faltas (L1083, L1093, L1115) e "janela encerrada não consome tentativa" (L763).

5.2 Requisitos: algum critério ficou sem teste? Algum teste ficou sem critério?

Na v1, cada um dos 21 critérios tem de 2 a 17 testes (CA02.2 é o mais testado). Todos os 117 testes trazem um comentário com um critério existente, sem nenhuma referência a critério inventado. Esse vínculo é declarado pela própria LLM; conferimos o conteúdo dos 48 testes ligados à matriz, mas não auditamos um a um os 69 extras.

5.3 Linhas versus branches

Uma linha conta como coberta se qualquer código dela executa; uma decisão na mesma linha tem dois caminhos, e basta um teste passar por um deles. Aqui isso não aparece: a v1 e a v2 têm **100% de linhas e 97,91% de branches**, e a única branch descoberta é a mesma nas duas, a linha 29 de `chamada-codigo.service.ts`, onde só há `@Injectable()`. A contagem `[0, 1]` de uma expressão condicional sem ternário no código-fonte indica código gerado na transpilação do decorator (inferência nossa). Não há decisão de negócio ali, e não identificamos teste que a exercite.

5.4 Oráculos

- **Sem asserção:** 0 de 117 testes (conferido por script).
- **Asserção genérica: 15 testes** usam `rejects.toThrow()` sem dizer o erro (L365, 433, 643, 650, 659, 673, 675, 696, 722, 733, 741, 750, 759, 766 e 843). O requisito pede erros específicos (400 para código incorreto, 409 para janela encerrada ou expirada, 429 para a sexta tentativa), mas esses testes passariam com qualquer exceção, inclusive um erro de programação. O teste da L679 compara as mensagens da 5ª e da 6ª tentativa, mas não os status.
- **Oráculo copiado do código:** não encontramos caso em que o código divergisse do critério, então o resultado dos testes não permite separar "oráculo do requisito" de "oráculo do código". Conferimos à mão as fronteiras sensíveis (5ª/6ª tentativa, instante exato da expiração, 15% e 25% de faltas) e o código segue o texto dos critérios. Nenhum teste valida um defeito conhecido.
- **Acoplamento à implementação:** 11 testes inspecionam os argumentos das chamadas ao Prisma. Com mocks isso é em grande parte inevitável (é a única forma de observar a gravação), mas L894, L906 e L930 verificam a forma interna da consulta (`where`, `select`), e não o efeito observável. Uma consulta equivalente quebraria esses testes sem mudar o comportamento. Um teste de integração com banco real evitaria isso.
- **Na v2:** T24, T26, T27 e T28 verificam o status HTTP (400, 429, 409 e 409) e o T28 também a mensagem; nenhum teste usa `toThrow()` genérico.

6. Melhorias da v2

Lacuna da v1	O que a v2 faz	Efeito medido
Oráculo genérico: 15 testes só verificam que "algo foi lançado"	T24, T26, T27 e T28 verificam o status HTTP do erro (e o T28 a mensagem)	0 oráculos genéricos (a v1 tem 15)

Transparência: a v2 foi escrita em 08/11 a partir da matriz, antes de a v1 existir. Contra a v1, a v2 **não aumenta a cobertura** (empate em 100% e 97,91%), só melhora a precisão do oráculo, e ela **não incorpora** os 69 testes extras da v1. A meta "branches acima da v1" **não foi atingida**, pois a única branch descoberta não é alcançável por teste (seção 5.3).

7. Conclusão

Uma LLM gera depressa uma suíte ampla: em uma rodada, sem correção, a v1 cobriu 100% das linhas e dos casos da matriz. Usaríamos uma LLM para **enumerar partições e valores-limite e montar o esqueleto dos testes**, com estes cuidados: (1) dar a ela os critérios completos e pedir as técnicas; (2) gerar numa conversa limpa, sem acesso ao gabarito; (3) revisar os oráculos, trocando `toThrow()` genérico por erro e status específicos; (4) medir com ferramenta, lembrando que 100% de linhas não significa que todas as decisões foram testadas; (5) complementar com testes de integração, pois mocks não provam concorrência, transações nem fraude presencial. O melhor resultado viria de **juntar a amplitude da v1 com a precisão de oráculo da v2**.

Apêndices (no repositório): `docs/testes/llm-log.md` (prompts e respostas), `docs/testes/prompt-v1.md`, `docs/testes/matriz-rastreabilidade.md` e `docs/testes/relatorios/`.